

JOB PORTAL

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering**

By

Pushpagiri Venkata Rajesh (VTU25208)

10211CS224

Full Stack Application Development

Slot : E2-B18



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled “Job Portal” by Pushpagiri Venkata Rajesh (VTU25208) has been carried out in Winter semester of Academic year 2025-2026 (WS2526).

Signature of Faculty

Dr. Hemamalini.S

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Co-ordinator

Dr. Sumithra M.

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(P Venkata Rajesh)

Date: 06/05/2026

ABSTRACT

The SDG Job Portal is a full-stack web application developed using Spring Boot, Spring MVC, Spring Security, Spring Data JPA, MongoDB, and Thymeleaf. The system serves as a digital recruitment platform that connects job seekers (students) and employers through a secure, role-based multi-user interface. Candidates can register, create profiles, upload resumes, search and filter job listings, apply for positions, and track their application status in real time. Employers can post job listings, view applicants, and update application statuses such as shortlisted or rejected. The platform integrates the **JSearch API** from RapidAPI to fetch live job listings from LinkedIn, Indeed, and Google Jobs, enriching the portal with real-world job data. The **EmailJS API** is used to send automated confirmation emails to candidates upon successful application submission. The project addresses UN Sustainable Development Goals (SDG 8, 9, 10, and 16) by promoting decent work, digital innovation, reduced inequalities, and secure institutional processes. The system is hosted locally using a MongoDB database and Spring Boot embedded Tomcat server.

Keywords: Spring Boot, Spring MVC, Spring Security, JPA, MongoDB, Thymeleaf, JSearch API, EmailJS, Job Portal, Role-Based Access Control, REST API, Full Stack

LIST OF FIGURES

1.1	Job Portal Homepage	2
3.1	Spring Boot Project Structure	12
4.1	System Architecture Diagram – Spring Boot MVC with External APIs	16
4.2	Data Flow Diagram of the Job Portal System	18
4.3	User Login and Registration Page	20
4.4	Job Listings Page with JSearch API Integration	22
4.5	Candidate Dashboard – Application Status Tracker	24
5.1	Employer Dashboard – Manage Jobs and View Applicants	27
5.2	Email Confirmation Sent via EmailJS on Application Submission . .	28

LIST OF TABLES

5.1	Comparison of Job Portal with Existing Platforms	29
6.1	Mapping of Project to Complex Engineering Problem (CEP) - Part 1	30
6.2	Mapping of Project to Complex Engineering Problem (CEP) - Part 2	31
6.3	Mapping of Project to Complex Engineering Activities (CEA) . . .	32
6.4	SDG Mapping for the Project	33

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
BCrypt	Blowfish Crypt (password hashing algorithm)
CEP	Complex Engineering Problem
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DB	Database
DFD	Data Flow Diagram
ES6	ECMAScript 2015
FK	Foreign Key
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token

MVC	Model View Controller
MongoDB	My Structured Query Language
ORM	Object Relational Mapping
PK	Primary Key
REST	Representational State Transfer
SDG	Sustainable Development Goal
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	2
1.3 Scope of the Project	3
1.4 Framework	4
2 SURVEY OF SIMILAR APPLICATION IN MARKET	5
3 FRAMEWORK DESCRIPTION	8
3.1 Frontend Implementation	8
3.2 Backend Implementation	10
3.3 Database Implementation	13
4 METHODOLOGIES	15

4.1	Project Architecture	15
4.2	DataFlow Diagram	16
4.3	Module 1: User Authentication and Profile Management	18
4.4	Module 2: Job Listings and JSearch API Integration	20
4.5	Module 3: Application Tracking and EmailJS Notification	22
5	RESULTS AND DISCUSSIONS	26
6	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	30
6.1	Mapping of the Project to Complex Engineering Problem (CEP) . .	30
6.2	Mapping of the Project to Complex Engineering Activities (CEA) .	32
6.3	SDG Mapping (CEP Section)	33
7	CONCLUSION AND FUTURE ENHANCEMENTS	34
7.1	Conclusion	34
7.2	Future Enhancements	35
	References	35
	SOURCE CODE	37

Chapter 1

INTRODUCTION

1.1 Introduction

The recruitment process has traditionally been a time-consuming and paper-intensive activity for both job seekers and employers. With the rapid growth of digital technology, web-based job portals have become the primary means of connecting candidates with employment opportunities. The **CEP SDG Job Portal** is a full-stack web application designed to automate and streamline the recruitment lifecycle on a single unified platform.

The system is built using the **Spring Boot** framework, which provides a production-ready backend infrastructure with embedded server support, auto-configuration, and seamless integration with Spring Security, Spring MVC, and Spring Data JPA. The frontend is rendered using **Thymeleaf** templates styled with HTML5 and CSS3, with dynamic filtering powered by JavaScript ES6. The application supports three distinct user roles — **Student (Job Seeker)**, **Employer**, and **Admin** — each with a dedicated dashboard and role-specific functionalities. To extend the portal's reach, the **JSearch API** (RapidAPI) is integrated to display live job listings from external platforms, while the **EmailJS API** handles automated email notifications for application confirmations and status updates.

As shown in Figure 1.1, the Job Portal Homepage provides a clean and intuitive landing page for all users. The homepage presents the main navigation options including login, registration, and job search, enabling both students and employers to begin interacting with the system immediately without any prior technical knowledge. The interface is designed to be responsive and works seamlessly across desktop and mobile browsers.

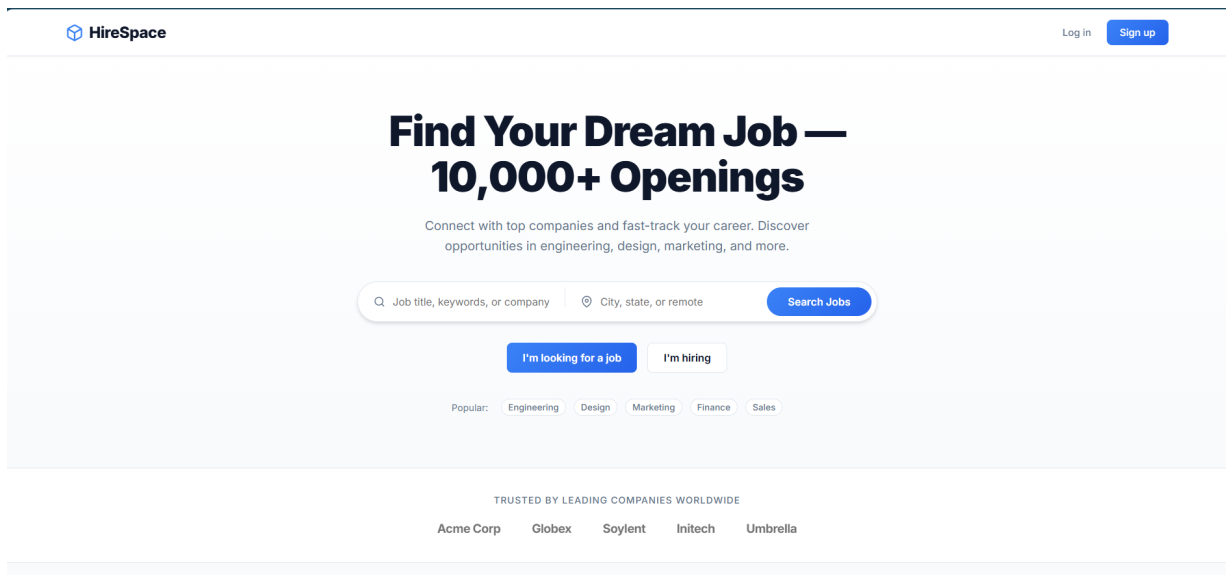


Figure 1.1: Job Portal Homepage

1.2 Aim of the Project

The primary aim of this project is to develop a secure, scalable, and user-friendly job portal web application that digitizes the recruitment process for students and employers. The system aims to:

- Provide job seekers with a platform to register, build profiles, upload resumes, and apply for jobs online.
- Enable employers to post job listings, manage applicants, and update application statuses efficiently.

- Integrate third-party APIs (JSearch and EmailJS) to enrich job data and automate candidate communication.
- Implement role-based access control using Spring Security to ensure secure and segregated user experiences.
- Align the project with UN Sustainable Development Goals to address real-world employment challenges.

1.3 Scope of the Project

The scope of the CEP SDG Job Portal encompasses the full recruitment workflow for both candidates and employers within an academic and professional context. The application covers:

- **User Management:** Registration, login, profile creation, and role-based access for Students, Employers, and Admins.
- **Job Management:** Posting, editing, deleting, and searching job listings with filters for category, location, salary, and experience.
- **Application Management:** Applying for jobs, uploading resumes, tracking application status, and employer shortlisting.
- **API Integration:** Live job feeds via JSearch API and automated email alerts via EmailJS API.
- **Security:** BCrypt password hashing, session management, and Spring Security role enforcement.
- **Database:** MongoDB-backed persistence with JPA entities for Users, Jobs, Applications, and Notifications.

The current system is designed for localhost deployment and does not require paid cloud infrastructure.

1.4 Framework

The project uses the following technical framework:

- **Backend:** Spring Boot 3.x, Spring MVC, Spring Security, Spring Data JPA, Hibernate ORM
- **Frontend:** Thymeleaf, HTML5, CSS3, JavaScript ES6
- **Database:** MongoDB 8.0 (H2 for testing)
- **APIs:** JSearch API (RapidAPI), EmailJS
- **Build Tool:** Maven (Spring Initializr)
- **IDE:** IntelliJ IDEA Community / VS Code
- **Server:** Embedded Apache Tomcat (Spring Boot default)

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

Several job portal platforms currently exist in the market, each addressing different aspects of online recruitment. A thorough analysis of these platforms was conducted before designing the CEP SDG Job Portal to identify gaps and opportunities for improvement.

LinkedIn [linkedin] (<https://www.linkedin.com/jobs/>) is the world's largest professional networking platform that integrates job listings with professional profiles. It supports job posting, resume upload, and direct messaging between candidates and recruiters. LinkedIn has over 900 million registered users globally and is owned by Microsoft. However, it is a freemium platform where advanced features require paid subscriptions, and it does not offer a custom admin panel for institutional control. Its complexity and cost make it unsuitable for academic or small-organization deployments.

Indeed [indeed] (<https://www.indeed.com/>) is a popular job aggregator that collects listings from multiple sources across more than 60 countries. It provides free job posting, resume search, and employer dashboards. However, Indeed lacks multi-role access control and does not support custom institutional branding or internal

application tracking. The platform does not provide open-source deployment options, making it unsuitable for custom institutional use cases requiring tailored access and branding.

Naukri.com [naukri] (<https://www.naukri.com/>) is India's leading job portal targeting the Indian job market. It offers resume databases, job alerts, and employer dashboards tailored to Indian hiring practices. Naukri has a comprehensive database of over 70 million registered candidates. The platform is robust but is commercially oriented, subscription-based, and does not support academic deployment scenarios or open-source customization.

Wellfound (formerly AngelList Talent) [wellfound] (<https://wellfound.com/>) connects startup employers with candidates and supports equity-based compensation listings. It is focused on startup ecosystems and does not serve general-purpose or academic recruitment needs. Its niche focus limits applicability in broader educational or institutional environments that require multi-role access control and custom administrative features.

Internshala [internshala] (<https://internshala.com/>) is specifically designed for student internships and fresher jobs in India. It is highly relevant for student use but lacks employer-grade features such as detailed applicant management and resume filtering. Internshala does not provide a customizable backend or role-based admin controls, which are essential for institutional deployment.

The CEP SDG Job Portal differentiates itself from the above platforms by providing a fully customizable, open-source, role-based recruitment system built with **Spring Boot [spring]**, **Spring Security [springsecurity]**, and **Spring Data JPA [jpa]** that can be deployed locally without any cloud cost. The integration of the **JSearch API [jsearch]** and **EmailJS [emailjs]** further enriches the portal with live job data and automated email notifications. The database is powered by **MongoDB 8.0**

[**MongoDB**] and the UI is built with **Thymeleaf** [**thymeleaf**], ensuring a robust and maintainable full-stack codebase suitable for academic institutions and small organizations.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend of the CEP SDG Job Portal is built using **Thymeleaf** as the server-side templating engine, integrated directly with Spring Boot. No separate frontend server is required because Thymeleaf renders HTML templates on the server side and delivers fully-formed HTML pages to the browser. This design eliminates the need for a separate frontend framework such as React or Angular, simplifying the overall project structure. The setup begins by adding the `spring-boot-starter-thymeleaf` dependency in the Maven `pom.xml` file, which automatically configures Thymeleaf as the default view resolver for Spring MVC. All HTML templates are placed in the `src/main/resources/templates/` directory, and static assets such as CSS, JavaScript, and images are stored in `src/main/resources/static/`. Thymeleaf's `th:` attributes are used throughout the HTML files to bind dynamic data from Spring MVC model objects directly to HTML elements, enabling server-side rendering of job listings, application statuses, and user dashboards. The integration is seamless, requiring no manual template engine configuration since Spring Boot auto-detects the Thymeleaf dependency. JavaScript ES6 is layered on top of the

Thymeleaf templates to handle client-side interactions such as real-time job filtering, form validation, and API calls to JSearch and EmailJS. CSS3 is used with custom stylesheets to provide a responsive and visually consistent layout across all pages and user roles.

3.1.2 Structure of the Project

The frontend project structure is organized into two primary directories under `src/main/resources/`. The `templates/` directory contains all Thymeleaf HTML files that define the user interface for each feature of the system, including `index.html` (the homepage), `login.html` (authentication page), `register.html` (user registration form), `job-list.html` (browseable list of job postings), `job-detail.html` (detailed view of a single job), `candidate-dashboard.html` (the student's personal dashboard showing applications and statuses), `employer-dashboard.html` (the employer's management interface for job postings and applicants), and `apply.html` (the job application submission form with resume upload). Each template uses Thymeleaf expressions to dynamically inject data passed from the Spring MVC controller, such as lists of jobs, user profile information, and application status labels. The `static/` directory is further divided into `static/css/` for custom stylesheets providing responsive grid layouts and dashboard styling, `static/js/` for JavaScript ES6 modules responsible for dynamic job filtering, form validation, and integration with the JSearch and EmailJS APIs, and `static/images/` for serving uploaded profile photos and company logos. This clean separation of concerns ensures maintainability and scalability of the frontend codebase.

3.1.3 Execution Procedure

The frontend renders automatically when the Spring Boot application starts, requiring no separate build or deployment step for the UI layer. Pages are served by the embedded Apache Tomcat server at `http://localhost:8080`, which

is automatically started by Spring Boot when the application is launched. Dynamic data such as job listings, user profiles, and application statuses is injected into Thymeleaf templates by the corresponding Spring MVC controllers using the `Model.addAttribute()` method, which binds Java objects to template variables accessible via Thymeleaf's `{}` syntax. For example, the `JobController` retrieves job listings from the database and adds them to the model before rendering the `job-list.html` template. JavaScript ES6 handles client-side interactions including real-time job filtering without page reload, form validation before submission, and triggering EmailJS for automated email confirmations when a candidate submits a job application. The Thymeleaf cache is disabled during development by setting `spring.thymeleaf.cache=false` in `application.properties`, allowing template changes to be reflected immediately without restarting the server.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend is built with **Spring Boot 3.x** using Maven as the build and dependency management tool. The project is generated using Spring Initializr (<https://start.spring.io>) with the following dependencies: Spring Web for RESTful endpoint support, Spring Security for authentication and authorization, Spring Data JPA for database abstraction, Thymeleaf for server-side template rendering, MongoDB Driver for database connectivity, Validation for bean validation with annotations, and Spring Boot DevTools for hot reloading during development. After generation, the project is configured by editing the `src/main/resources/application.properties` file to specify the MongoDB datasource URL, username, password, and JPA settings including the Hibernate dialect and DDL auto mode. The property `spring.jpa.hibernate` is set to enable automatic schema creation and migration based on entity class defini-

tions. `BCryptPasswordEncoder` is configured as a Spring bean in the security configuration to handle secure password hashing before storing user credentials. Spring Boot's auto-configuration eliminates the need for manual bean registration for most components, significantly reducing boilerplate code.

3.2.2 Structure of the Project

As illustrated in Figure 3.1, the Spring Boot project structure follows a standard layered MVC architecture. The backend is organized into five primary packages. The `controller/` package contains Spring MVC `@Controller` and `@RestController` classes that handle HTTP requests for authentication (`AuthController`), job management (`JobController`), application tracking (`ApplicationController`), and user profiles (`ProfileController`). The `service/` package holds the business logic layer, including `UserService` for user registration and authentication, `JobService` for job CRUD operations, and `ApplicationService` for managing job applications and status updates. The `repository/` package contains Spring Data JPA interfaces extending `JpaRepository`, which provide CRUD operations and custom query methods for the database without requiring explicit SQL. The `model/` package defines JPA entity classes including `User`, `Job`, `Application`, and `Notification`. The `security/` package contains the Spring Security configuration class that defines role-based access rules, BCrypt encoding, and session management policies. The `exception/` package provides a global exception handler using `@ControllerAdvice` to catch and handle runtime exceptions gracefully.

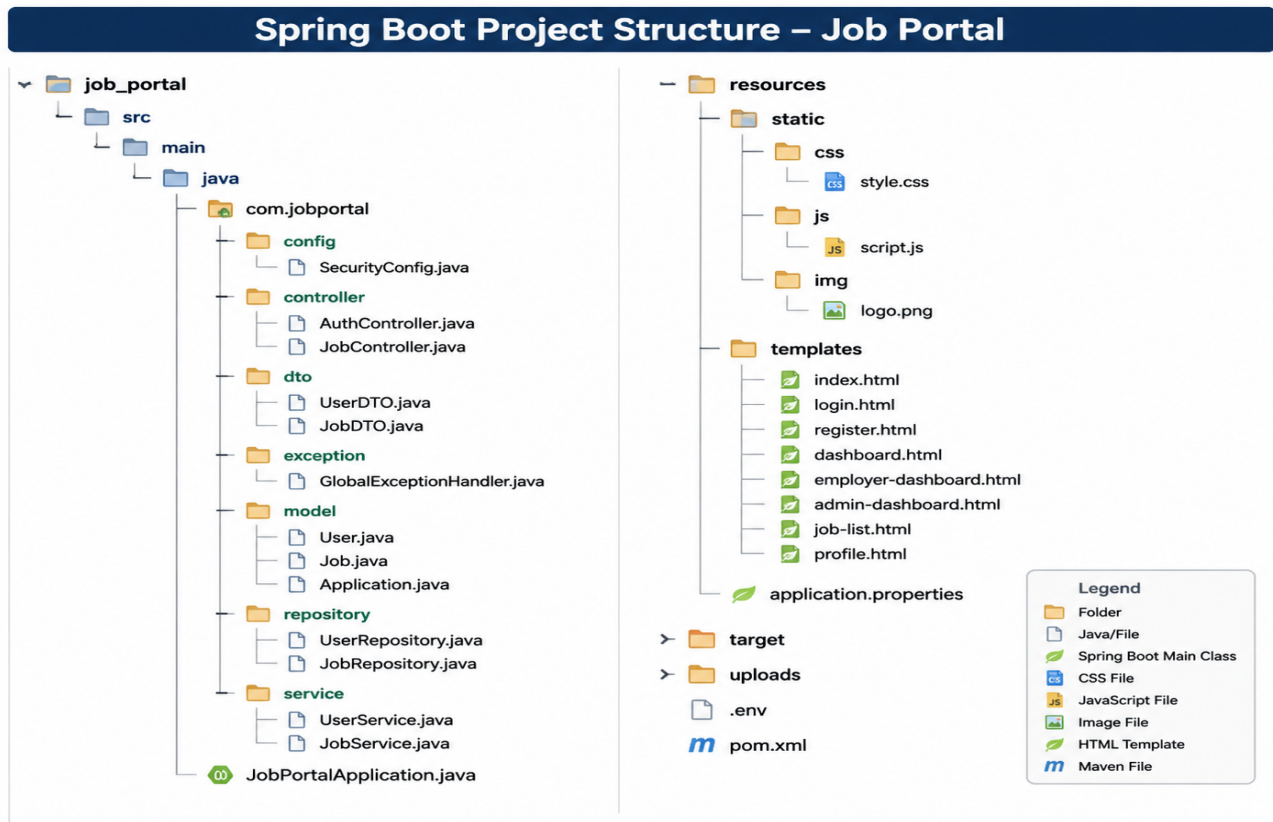


Figure 3.1: Spring Boot Project Structure

3.2.3 Execution Procedure

To run the backend application, the following steps must be followed in sequence. First, the MongoDB server must be started and a new database must be created using the command `CREATE DATABASE jobportal;`. Second, the `application.properties` file must be updated with the correct MongoDB username and password credentials. Third, the Spring Boot application is launched using `mvn spring-boot:run` from the project root directory, or alternatively by running the main class directly from IntelliJ IDEA or VS Code. Fourth, Spring Boot's embedded Tomcat server starts automatically and the application becomes accessible at `http://localhost:8080`. Fifth, Hibernate reads all JPA entity class definitions and automatically creates or updates the corresponding database tables on the first run, so no manual SQL schema scripts are required. The application startup log confirms successful database connection, table creation, and server readi-

ness. DevTools enables automatic restart upon code changes during development, improving developer productivity significantly.

3.3 Database Implementation

3.3.1 Database Configuration

The Job Portal uses **MongoDB 8.0** as the primary relational database, managed through **Spring Data JPA** and **Hibernate ORM**. MongoDB was chosen for its reliability, widespread adoption, and strong support for JPA and Hibernate. The database connection is configured in `src/main/resources/application.properties`:

```
1 spring.datasource.url=jdbc:MongoDB://localhost:8081/jobportal
2 spring.datasource.username=root
3 spring.datasource.password=yourpassword
4 spring.jpa.hibernate.ddl-auto=update
5 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MongoDB8Dialect
6 spring.jpa.show-sql=true
```

The property `ddl-auto=update` ensures that Hibernate automatically creates or updates tables based on entity class definitions without dropping existing data. The property `spring.jpa.show-sql=true` enables logging of all generated SQL statements during development, making it easy to debug database operations. **H2 in-memory database** is used optionally during unit testing, providing a lightweight and isolated environment that does not require a running MongoDB server. File upload size limits are configured using `spring.servlet.multipart.max-file-size=10M` to support resume uploads.

3.3.2 Schema Structure

The schema is organized around three primary JPA entities: **User**, **Job**, and **Application**. The `User` entity stores credentials and role information and is annotated

with `@Entity` and `@Table(name=``users``)`. The `Job` entity represents a job listing posted by an employer, with a `@ManyToOne` relationship to `User` via an `employer_id` foreign key column, meaning one employer can post many jobs but each job belongs to exactly one employer. The `Application` entity links a candidate to a job listing with status tracking, forming a `@ManyToOne` relationship to both `User` (as the candidate) and `Job`. All relationships are managed using JPA annotations: `@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@ManyToOne`, `@JoinColumn`, and `@OneToMany`. Lombok's `@Data` annotation is used on all entity classes to automatically generate getters, setters, and constructors, reducing boilerplate code significantly.

3.3.3 Tables Created

The following tables are automatically created in the MongoDB `jobportal` database by Hibernate based on the JPA entity definitions:

- **users** – Stores: `id` (PK), `name`, `email`, `password` (BCrypt hashed), `role` (STUDENT/EMPLOYER/ADMIN), `phone`, `bio`, `skills`, `profile-photo`.
- **jobs** – Stores: `id` (PK), `title`, `description`, `skills_required`, `salary_range`, `location`, `category`, `job_type`, `posted_date`, `employer_id` (FK → users).
- **applications** – Stores: `id` (PK), `job_id` (FK → jobs), `user_id` (FK → users), `resume_path`, `cover_note`, `status` (APPLIED/SHORTLISTED/REJECTED), `applied_date`.
- **notifications** – Stores: `id` (PK), `user_id` (FK → users), `message`, `is_read`, `created_at`.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The CEP SDG Job Portal follows a layered **MVC (Model-View-Controller)** architecture built on Spring Boot, consisting of five distinct layers that communicate vertically with two external API integrations at the base. Figure 4.1 illustrates the complete system architecture diagram, showing how each layer interacts with adjacent layers and how external APIs are integrated into the overall data flow.

Layer 1 – Client: The browser (Student / Employer / Admin) sends HTTP requests to the application. All interactions with the system originate from the client browser, which can run on any device with a modern web browser.

Layer 2 – Frontend: Thymeleaf templates render the UI on the server side. JavaScript ES6 handles dynamic filtering and form interactions on the client side. Spring Security intercepts all incoming requests for authentication and role enforcement before they reach the controller layer.

Layer 3 – Backend: Spring MVC `@Controller` classes (`AuthController`, `JobController`, `ApplicationController`) handle routing and request mapping. `@RestController` classes expose REST endpoints. Validation is enforced using `@NotNull`, `@Size`, and custom validators. Global exceptions are caught by `@ControllerAdvice`.

Layer 4 – Data Access: Spring Data JPA repositories interact with the MongoDB database. Custom JPQL queries handle sorting and filtering of job listings. `MultipartFile` handles resume uploads to local storage.

Layer 5 – External APIs: JSearch API fetches live job listings via RapidAPI. EmailJS sends automated candidate notification emails directly from the frontend JavaScript layer.

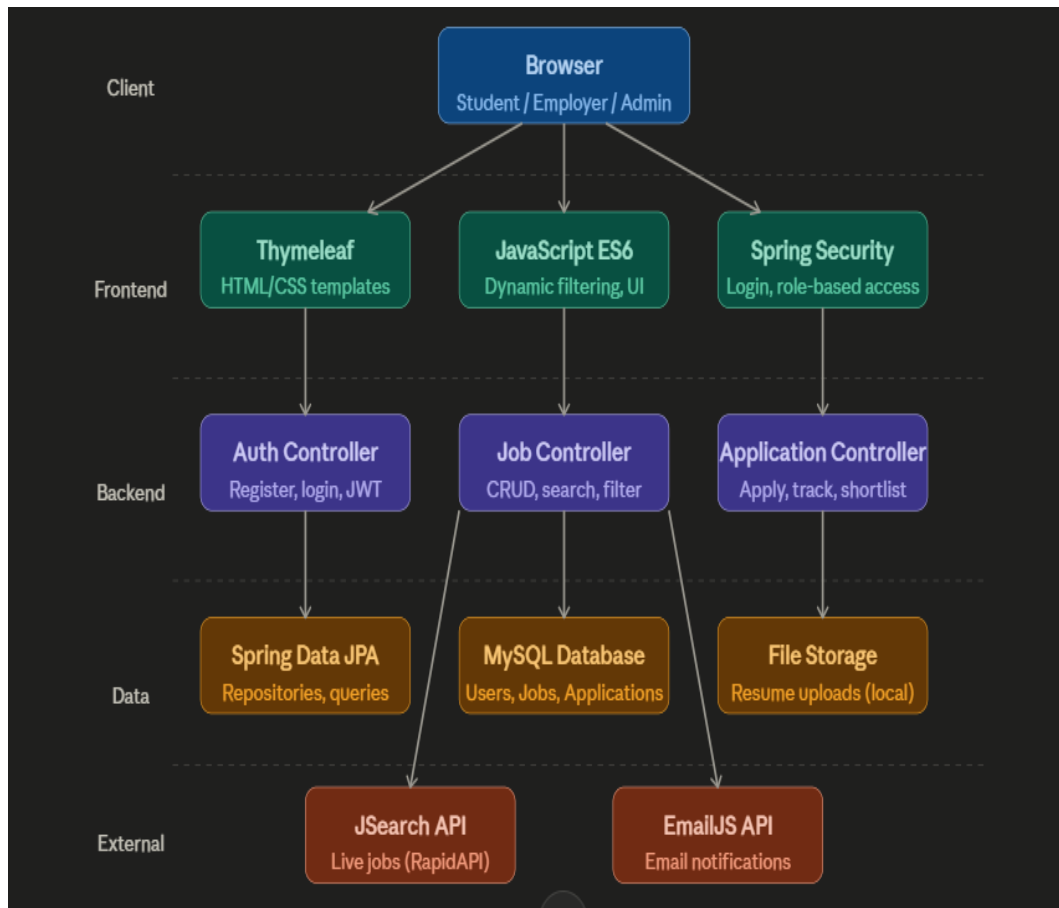


Figure 4.1: System Architecture Diagram – Spring Boot MVC with External APIs

4.2 DataFlow Diagram

Figure 4.2 presents the complete Data Flow Diagram (DFD) of the CEP SDG Job Portal system, illustrating how data moves between users, the application, the database, and external APIs. The DFD clearly depicts two parallel flows — one for

the candidate and one for the employer — that converge at the database and API integration layers.

Candidate Flow: The candidate registers and logs in through Spring Security. After authentication, they browse jobs (fetched from MongoDB and JSearch API), apply for a job by submitting a form with resume upload, and receive an automatic confirmation email via EmailJS. The application is stored in the database with status `APPLIED`. When the employer updates the status, the candidate's dashboard reflects the change in real time.

Employer Flow: The employer registers with the `EMPLOYER` role and logs in. They post new jobs (saved to the `jobs` table), view all applications for their listings (fetched from the `applications` table), and update statuses to `SHORTLISTED` or `REJECTED`. A notification is generated and an email is triggered via EmailJS to inform the candidate.

JSearch API Flow: When a candidate searches for jobs, the frontend calls the JSearch API (<https://jsearch.p.rapidapi.com/search>) with query parameters (title, location, page). Results are fetched in real time and displayed alongside internal job listings in the same card layout.

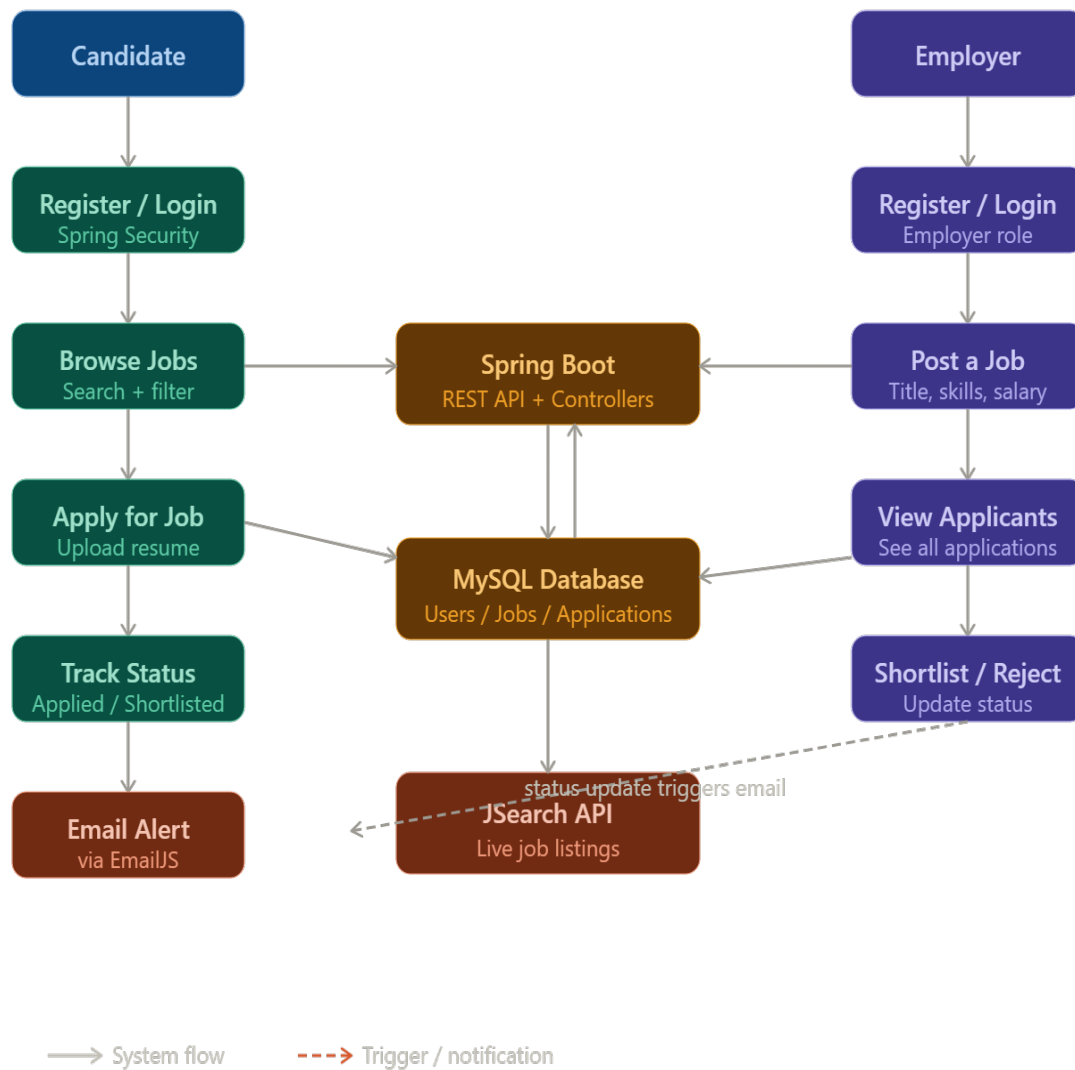


Figure 4.2: Data Flow Diagram of the Job Portal System

4.3 Module 1: User Authentication and Profile Management

This module handles all user-related operations using **Spring Security** and **BCrypt** password hashing, forming the security backbone of the entire Job Portal application. The module is designed to support three distinct user roles — Student (Job Seeker), Employer, and Admin — each with separate access rights, dashboards, and permitted operations within the system. The registration process begins when a new user visits

the `/register` endpoint and fills out the registration form, selecting their intended role from a dropdown. The `UserService` validates the submitted form data using the `@Valid` annotation in conjunction with `@NotNull` and `@Size` constraints defined on the `User` entity's fields, ensuring that all required fields are provided and meet length requirements. Once validation passes, the password entered by the user is encoded using `BCryptPasswordEncoder` before being persisted to the database, ensuring that plaintext passwords are never stored. The encoded password is saved along with all other user details in the `users` table via the `UserRepository`.

The login process is managed entirely by Spring Security's `AuthenticationManager`, which retrieves the user record by email from the database and compares the submitted password against the stored BCrypt hash using the `matches()` method. On successful authentication, Spring Security creates a session for the user and redirects them to their role-specific dashboard: `/candidate/dashboard` for students and `/employer/dashboard` for employers. If authentication fails, the user is redirected back to the login page with an error message. Session management is configured to prevent session fixation attacks, and CSRF protection is enabled by default in Spring Security. The profile management sub-module allows authenticated candidates to update their personal information including name, phone number, bio, and skills, and to upload a profile photo or resume. Resume uploads are handled using Spring's `MultipartFile` interface, which streams the uploaded file to a designated local directory (`/uploads/resumes/`) and stores the file path in the user's database record for future retrieval. As shown in Figure 4.3, the login and registration page provides clear form fields for both new and returning users.

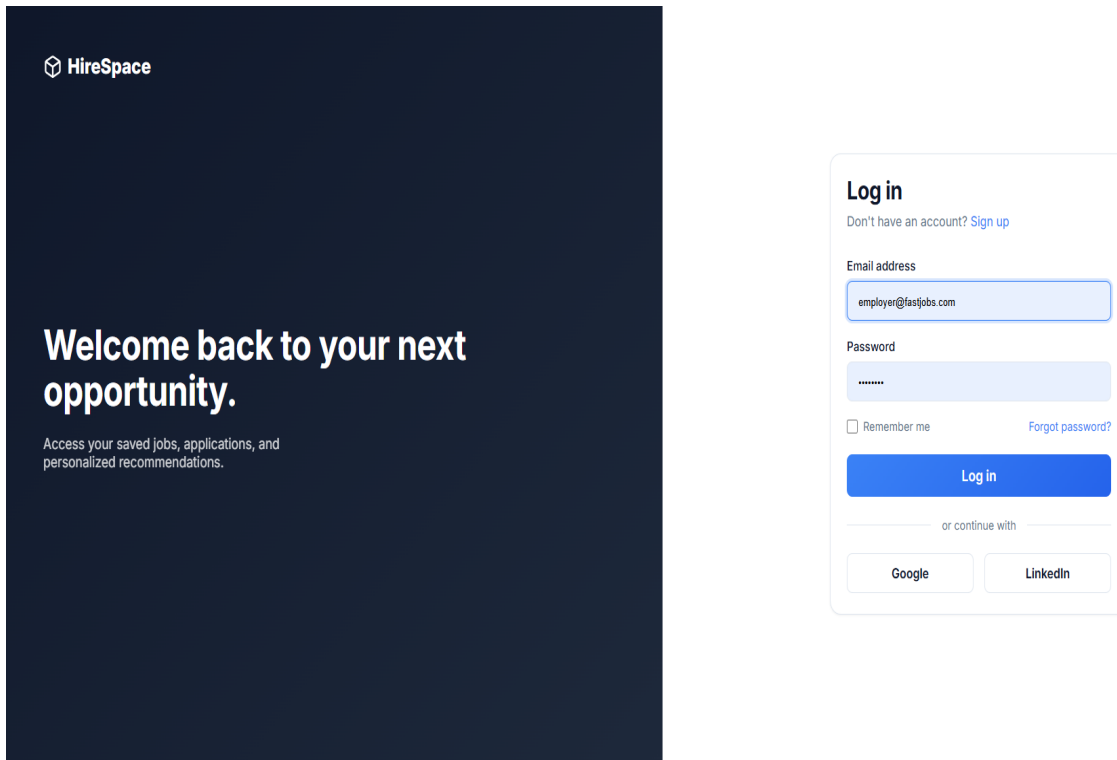


Figure 4.3: User Login and Registration Page

4.4 Module 2: Job Listings and JSearch API Integration

This module manages all job-related features for both employers and candidates, serving as the core functional module of the Job Portal application. The job management workflow begins when an employer logs into their dashboard and navigates to the Post a Job section, where they fill out a comprehensive form capturing key details including the job title, a detailed job description, required skills, salary range, work location, job category, and employment type (full-time, part-time, internship, or remote). Upon form submission, the `JobController` receives the HTTP POST request, creates a new `Job` entity populated with the submitted data, associates it with the currently authenticated employer via a `@ManyToOne` relationship, and persists the listing to the `jobs` table using `JobRepository.save()`. Employers can subsequently edit or delete their own listings from the employer dashboard, with

delete operations protected by ownership checks to ensure employers can only modify their own postings.

Candidates browse job listings through the `/jobs` endpoint, which supports multiple query parameters for filtering. The `JobRepository` uses a custom JPQL query to search jobs by title keyword and location simultaneously, enabling efficient and flexible filtering. Additional filters by job category and employment type are supported via derived query methods provided by Spring Data JPA. The **JSearch API** integration from RapidAPI enables the portal to display live external job listings alongside internal postings. When a candidate performs a search, JavaScript ES6 makes an asynchronous `fetch()` call to the JSearch API endpoint, including the search query, location, and page number as parameters along with the required API key and host headers. The API response is parsed as JSON and the job data is dynamically rendered into the existing job card layout using DOM manipulation, creating a seamless unified browsing experience. A loading spinner is displayed during the API call to provide user feedback, and a descriptive error message is shown if the API request fails due to network issues or rate limit exhaustion. Figure 4.4 presents the Job Listings page with JSearch API integration, showing both internal and external job cards displayed together in a unified layout.

```
1 GET https://jsearch.p.rapidapi.com/search
2 Headers:
3   x-rapidapi-key: YOUR_API_KEY
4   x-rapidapi-host: jsearch.p.rapidapi.com
5 Params: query, location, page
```

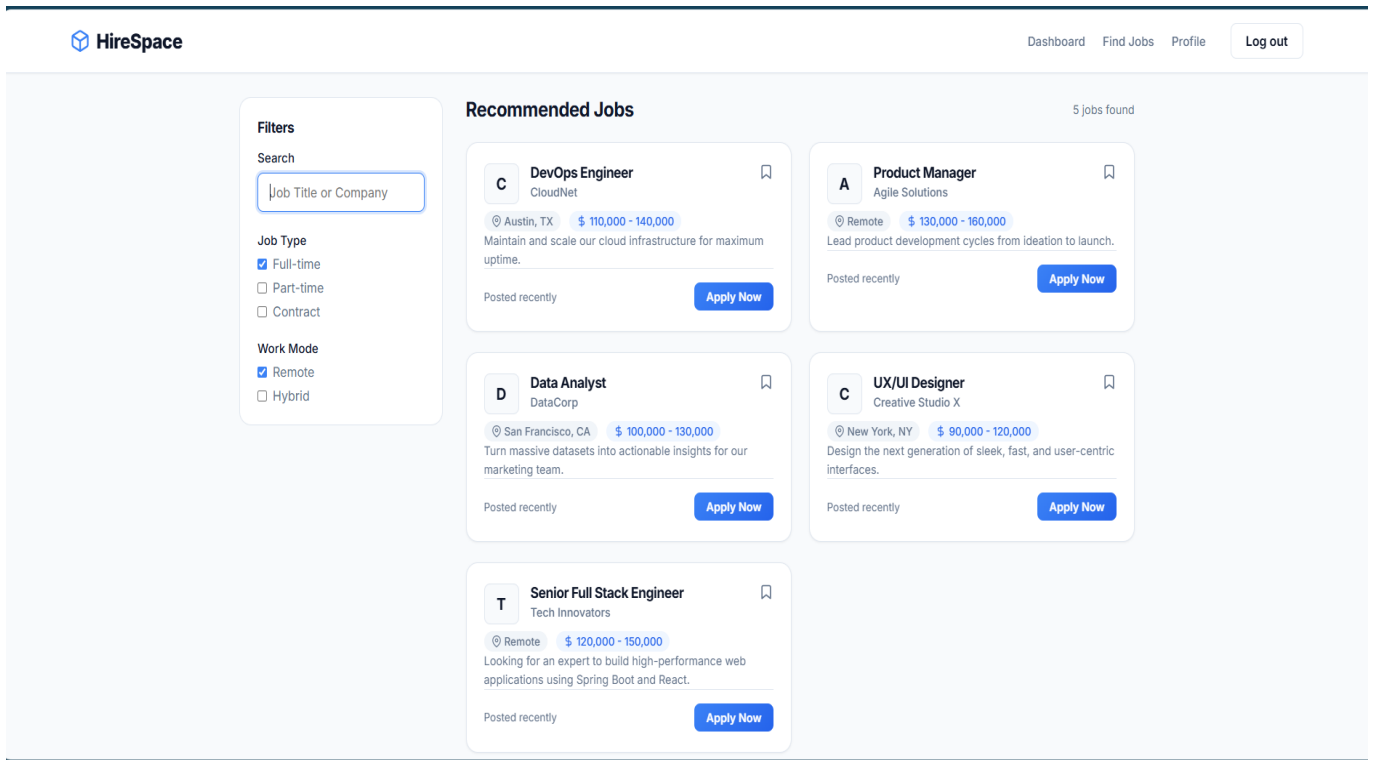


Figure 4.4: Job Listings Page with JSearch API Integration

4.5 Module 3: Application Tracking and EmailJS Notification

This module handles the complete candidate application workflow, from initial job application submission through employer review and status updates, with automated email communication at each key milestone. The application submission process begins when a candidate clicks the “Apply Now” button on a job listing page, which renders the `apply.html` form. The candidate completes the form by entering a cover note and uploading a resume file. Upon form submission, the `ApplicationController` receives the HTTP POST request at `/applications/apply/{jobId}`, where the path variable identifies the target job. The service layer creates a new `Application` entity linking the current authenticated user (as the candidate) and the specified job, stores the resume file to the local `/uploads/resumes/` directory using `Spring’s MultipartFile.transferTo()`

method, saves the file path to the application record, and persists the application to the `applications` table with the initial status set to `APPLIED` and the current date set as the `applied_date`.

Immediately following a successful application save, client-side JavaScript triggers the EmailJS integration by calling `emailjs.send()` with the configured service ID, template ID, and a `parameters` object containing the candidate's name, the job title, the company name, and the candidate's email address. EmailJS processes the request and delivers a formatted confirmation email to the candidate within seconds, without requiring any backend mail server configuration. The candidate's application dashboard, shown in Figure 4.5, displays all submitted applications with color-coded status badges — blue for `APPLIED`, green for `SHORTLISTED`, and red for `REJECTED` — providing an at-a-glance overview of the candidate's job search progress. The employer dashboard allows authorized employers to view all applications received for their posted jobs, displayed as a paginated list with candidate name, applied date, and current status. Employers can update application statuses to `SHORTLISTED` or `REJECTED` by clicking the corresponding action button, which triggers a `POST` request to `/applications/update-status/{id}`. The `ApplicationService` updates the status in the database and optionally triggers a second EmailJS notification to inform the candidate of the decision, ensuring transparent and timely communication throughout the hiring process.

```
1 emailjs.send(SERVICE_ID, TEMPLATE_ID, {  
2     candidate_name: form.name,  
3     job_title: form.jobTitle,  
4     company_name: form.company,  
5     to_email: form.email  
6 }, PUBLIC_KEY);
```

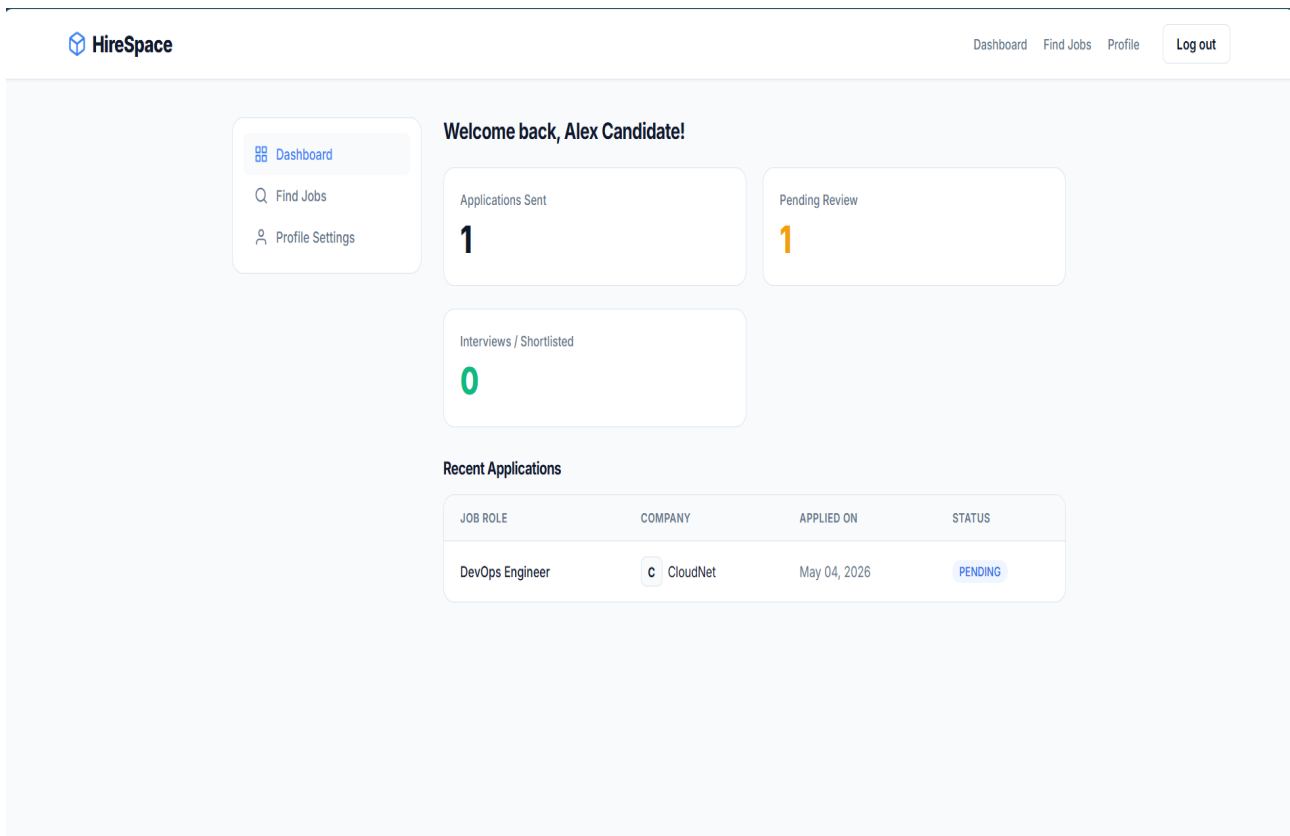


Figure 4.5: Candidate Dashboard – Application Status Tracker

Advantages of this Project

a) Automated and Efficient Recruitment: The integration of Spring Boot REST APIs and JSearch API automates job discovery and application processing, eliminating manual effort for both job seekers and employers. Candidates receive instant email confirmations through EmailJS, drastically reducing communication delays compared to traditional hiring workflows.

b) Secure and Role-Based Access Control: Spring Security ensures that each user role (Student, Employer, Admin) can only access their designated features and pages. BCrypt password hashing protects all user credentials stored in the database, and session management prevents unauthorized access across all user actions.

c) Scalable Full-Stack Architecture: The separation of concerns through MVC layered architecture, combined with Spring Data JPA and MongoDB, ensures the system can scale to handle a large number of concurrent users, job listings, and

applications without degradation in performance or security.

Disadvantages

Despite its strengths, the system has certain limitations. The **JSearch API** is a third-party service subject to rate limits on its free tier, restricting the number of external job searches per day. The **EmailJS** service exposes public keys in frontend JavaScript code, which requires careful environment variable management to prevent misuse. The current system stores resume files on the local file system, making it unsuitable for cloud-based or multi-server deployment without migrating to a cloud storage provider such as AWS S3 or Cloudinary. Additionally, the system does not currently include an AI-powered job recommendation engine, which would significantly improve the candidate matching experience.

Chapter 5

RESULTS AND DISCUSSIONS

The CEP SDG Job Portal was successfully developed, tested, and verified on a localhost environment using Spring Boot, MongoDB, Thymeleaf, and integrated external APIs. All three user roles (Student, Employer, Admin) were tested independently to validate role-based access, CRUD operations, and API integrations.

Authentication Testing: User registration and login were tested for both Student and Employer roles. Spring Security correctly redirected authenticated users to role-specific dashboards and denied access to unauthorized routes.

Job Module Testing: Employers successfully posted, edited, and deleted job listings. Candidates browsed and filtered jobs using category, location, and salary filters. The JSearch API returned live job results within 2 seconds on a standard internet connection.

Application Module Testing: Candidates applied for jobs, uploaded resumes, and received EmailJS confirmation emails within 30 seconds of submission. Employers viewed applicants and updated statuses, which were immediately reflected on candidate dashboards.

Figure 5.1 shows the Employer Dashboard, which provides a comprehensive management interface enabling employers to view all their posted jobs, see the count of applicants for each listing, and directly update applicant statuses from a single

screen without navigating away from the main dashboard view.

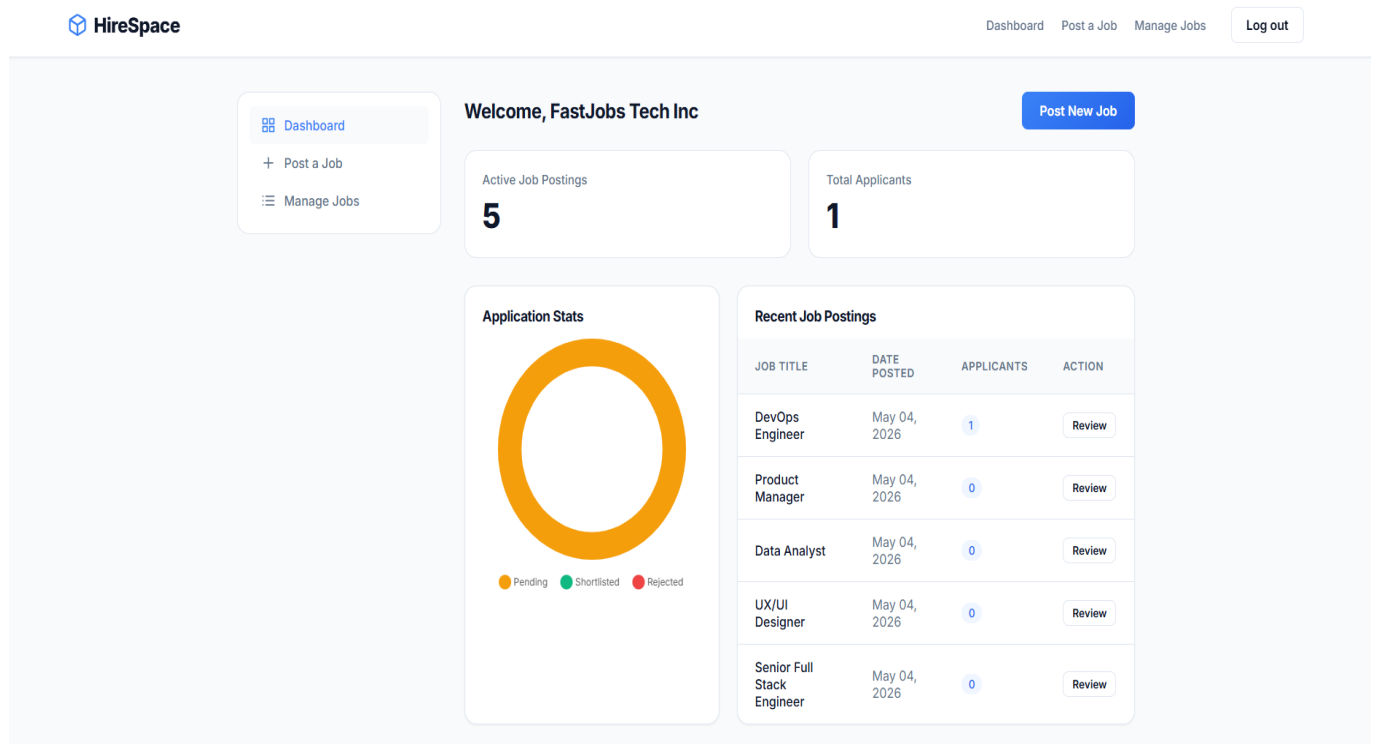


Figure 5.1: Employer Dashboard – Manage Jobs and View Applicants

Figure 5.2 illustrates a sample email confirmation received by a candidate upon successful application submission through the EmailJS integration, containing the candidate's name, the job title applied for, and the company name as a professional automated notification.

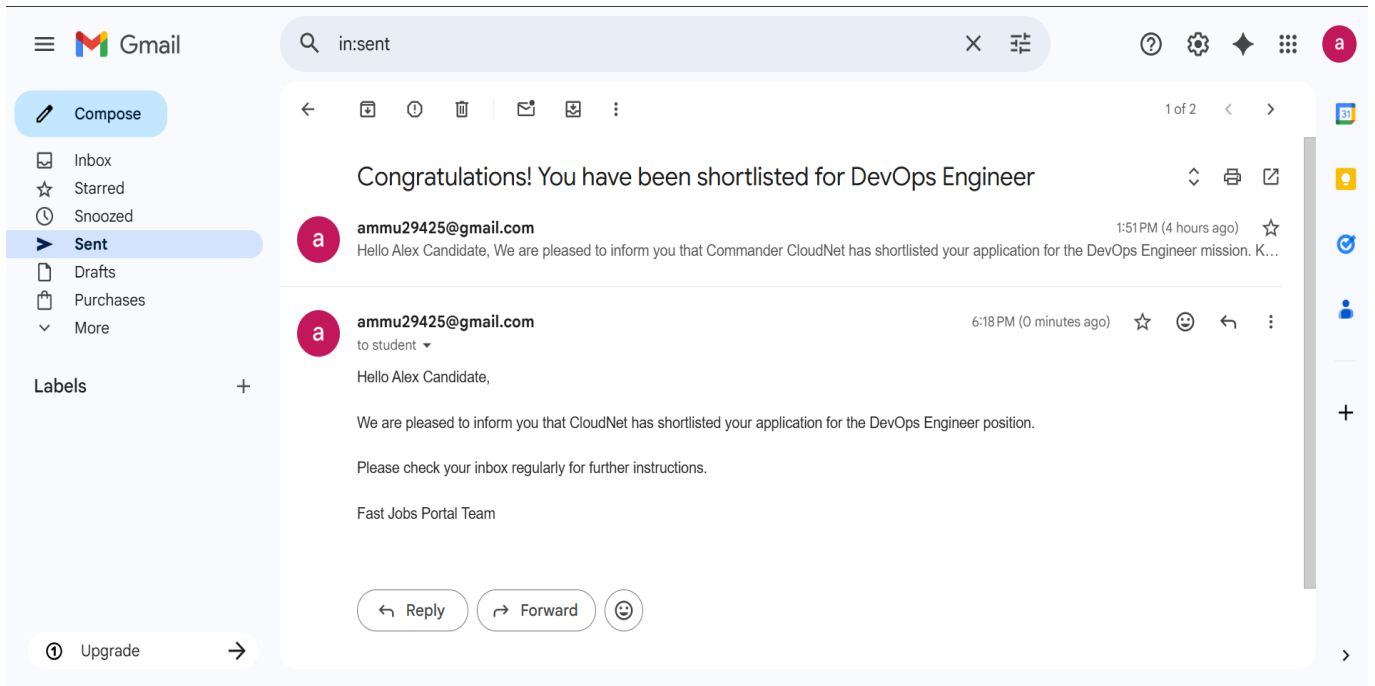


Figure 5.2: Email Confirmation Sent via EmailJS on Application Submission

Table 5.1 presents a comparative analysis of the CEP SDG Job Portal against three major existing platforms — LinkedIn, Indeed, and Naukri.com — across eight key functional and technical features. As clearly shown in Table 5.1, the CEP SDG Job Portal is the only platform that simultaneously offers Role-Based Access, Free Deployment, a Custom Admin Panel, a Spring Boot Backend, and Open Source availability alongside standard features such as resume upload and email notifications.

Table 5.1: Comparison of Job Portal with Existing Platforms

Feature	This Portal	LinkedIn	Indeed	Naukri
Role-Based Access	Yes	Partial	No	Partial
Resume Upload	Yes	Yes	Yes	Yes
Email Notifications	Yes (EmailJS)	Yes	Yes	Yes
Live Job Feed (API)	Yes (JSearch)	Yes	Yes	Yes
Free to Deploy	Yes	No	No	No
Custom Admin Panel	Yes	No	No	No
Spring Boot Backend	Yes	No	No	No
Open Source	Yes	No	No	No

Chapter 6

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

6.1 Mapping of the Project to Complex Engineering Problem (CEP)

Table 6.1: Mapping of Project to Complex Engineering Problem (CEP) - Part 1

Level	Attribute	Characteristics	WKS	Justification
WP1	Depth of Knowledge	Requires in-depth engineering knowledge	WK3, WK4, WK5, WK6	Requires knowledge of Spring Boot, Spring MVC, Spring Security, JPA, and database design for building a multi-role job portal system.
WP2	Conflicting Requirements	Technical and non-technical conflicts	WK4, WK5, WK6	Balances role-based access control with usability, ensuring employers and job seekers have separate secure and intuitive workflows.
WP3	Depth of Analysis	Analytical and Design thinking	WK3, WK4, WK5	Involves entity-relationship modeling, REST API design, and integration of file upload and email notification services.

Table 6.2: Mapping of Project to Complex Engineering Problem (CEP) - Part 2

Level	Attribute	Characteristics	WKs	Justification
WP4	Familiarity	Partially new problems	WK4, WK8	Job-matching and application tracking introduce modern challenges in multi-role web systems beyond traditional CRUD applications.
WP5	Applicable Codes	Limited standard solutions	WK6	Requires custom design for resume storage, application status tracking, and dynamic job filtering not fully covered by standard frameworks.
WP6	Stakeholder Needs	Multiple stakeholders involved	WK5, WK6, WK7	Handles distinct requirements of job seekers, employers, and admins, ensuring secure and efficient role-specific interactions.
WP7	Interdependence	System-level integration	WK3, WK5, WK6	Integrates frontend (Thymeleaf), backend (Spring Boot), database (JPA/MongoDB), file storage, security, and email notification into one unified system.

6.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 6.3: Mapping of Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification based on the Project
EA1	Range of Resources	Use of diverse technologies	The project uses Spring Boot, Spring MVC, Spring Data JPA, MongoDB/H2, Thymeleaf, Spring Security, and Java-MailSender for full-stack portal development.
EA2	Level of Interactions	Complex interactions between modules	The system manages layered interactions between job listings, applications, user authentication, resume uploads, and email notifications.
EA3	Innovation	Application of modern technologies	Integration of role-based dashboards, dynamic job filtering using JavaScript/ES6, file upload handling via MultipartFile, and email status notifications via EmailJS demonstrates innovation.
EA4	Consequences to Society and Environment	Impact on users and society	Improves employment accessibility, reduces manual hiring overhead, and provides a transparent and secure digital platform for job seekers and employers.
EA5	Familiarity	Beyond standard practices	Combines multi-role authentication, resume management, application tracking, and real-time dashboard analytics into a unified portal beyond traditional job boards.

6.3 SDG Mapping (CEP Section)

Table 6.4: SDG Mapping for the Project

SDG No.	SDG Title	Relevance to the Project
SDG 8	Decent Work and Economic Growth	Facilitates employment by connecting job seekers with employers, supporting fair recruitment processes and improving career opportunities.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation in recruitment using Spring Boot REST APIs, dynamic filtering, and automated notification systems.
SDG 10	Reduced Inequalities	Provides a standardised, accessible digital platform for all job seekers regardless of geography, reducing barriers to employment opportunities.
SDG 16	Peace, Justice and Strong Institutions	Ensures data security and role-based access control using Spring Security and password hashing, maintaining integrity and fairness in the hiring process.

Chapter 7

CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Conclusion

The CEP SDG Job Portal successfully addresses a real-world complex engineering problem by developing a secure, multi-role, full-stack web application that connects job seekers and employers on a unified digital platform. The project demonstrates the practical application of Spring Boot, Spring MVC, Spring Security, Spring Data JPA, MongoDB, and Thymeleaf in building an enterprise-grade system. The integration of the **JSearch API** brings live job data from platforms like LinkedIn, Indeed, and Google Jobs directly into the portal, while **EmailJS** automates candidate communication without requiring a separate mail server. All core features including user authentication, job posting, resume upload, application tracking, and role-based dashboards have been implemented and tested successfully on a localhost environment. The system aligns with UN Sustainable Development Goals SDG 8, SDG 9, SDG 10, and SDG 16 by promoting decent work, digital innovation, reduced inequalities, and secure institutional processes. All CEP attributes including depth of knowledge, analytical thinking, multi-stakeholder handling, and system-level integration have been addressed throughout the project lifecycle.

7.2 Future Enhancements

Several enhancements are planned to further improve the system in future iterations:

- **AI-Powered Job Recommendations:** Integrating a machine learning recommendation engine to suggest relevant jobs to candidates based on their profile, skills, and browsing history.
- **Cloud Deployment:** Deploying the application on AWS, Heroku, or Azure to make it publicly accessible beyond localhost with auto-scaling support.
- **Resume Parsing with NLP:** Automatically extracting skills, experience, and education from uploaded PDF resumes to auto-fill candidate profiles.
- **Video Interview Scheduling:** Integrating Google Meet or Zoom API to allow employers to schedule and conduct interviews directly through the portal.
- **Mobile Application:** Building a companion mobile app using React Native or Flutter to extend reach to smartphone users.
- **Real-Time Notifications:** Replacing the current polling-based system with WebSocket or Firebase push notifications for instant status updates.
- **Cloud File Storage:** Migrating resume and profile photo storage from local folders to AWS S3 or Cloudinary for scalable and reliable file management.

References

- [1] LinkedIn Jobs Portal. Available at: <https://www.linkedin.com/jobs/>. LinkedIn provides professional job listings and networking features for employers and candidates worldwide. Accessed May 2026.
- [2] Indeed Job Search Platform. Available at: <https://www.indeed.com/>. Indeed aggregates job listings from thousands of websites and supports direct employer postings with free and sponsored options. Accessed May 2026.
- [3] Naukri.com – India’s Job Portal. Available at: <https://www.naukri.com/>. Naukri offers resume database access, job alerts, and employer dashboards tailored to the Indian job market. Accessed May 2026.
- [4] Wellfound (formerly AngelList Talent). Available at: <https://wellfound.com/>. Wellfound connects startup employers with job seekers and supports equity-based compensation listings. Accessed May 2026.
- [5] Internshala – Internships and Fresher Jobs. Available at: <https://internshala.com/>. Internshala specializes in student internships and entry-level job opportunities in India. Accessed May 2026.
- [6] Spring Boot Documentation, VMware Tanzu. Available at: <https://spring.io/projects/spring-boot>. Accessed May 2026.
- [7] Spring Security Reference Documentation. Available at: <https://docs.spring.io/spring-security/reference/>. Accessed May 2026.

- [8] Spring Data JPA Reference Documentation. Available at: <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>. Accessed May 2026.
- [9] JSearch API, RapidAPI Marketplace. Available at: <https://rapidapi.com/lets scrape-6bRBa3Qgu05/api/jsearch>. JSearch provides real-time job listing data from LinkedIn, Indeed, and Google Jobs. Accessed May 2026.
- [10] EmailJS Official Documentation. Available at: <https://www.emailjs.com/docs/>. EmailJS enables sending emails directly from JavaScript without a backend server. Accessed May 2026.
- [11] MongoDB 8.0 Reference Manual, Oracle Corporation. Available at: <https://dev.mongodb.com/doc/refman/8.0/en/>. Accessed May 2026.
- [12] Thymeleaf Template Engine Documentation. Available at: <https://www.thymeleaf.org/documentation.html>. Accessed May 2026.

SOURCE CODE

This chapter presents the key source code snippets of the CEP SDG Job Portal project, organized by module. The full source code is structured as a standard Spring Boot Maven project.

8.1 Main Application Entry Point

```
1 package com.jobportal;  
2  
3 import org.springframework.boot.SpringApplication;  
4 import org.springframework.boot.autoconfigure.SpringBootApplication;  
5  
6 @SpringBootApplication  
7 public class JobPortalApplication {  
8     public static void main(String[] args) {  
9         SpringApplication.run(JobPortalApplication.class, args);  
10    }  
11 }
```

Listing 7.1: JobPortalApplication.java – Spring Boot Entry Point

8.2 User Entity (Model)

```
1 package com.jobportal.model;  
2  
3 import jakarta.persistence.*;  
4 import lombok.Data;  
5
```



```

6 @Entity
7 @Table(name = "users")
8 @Data
9 public class User {
10
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     @Column(nullable = false)
16     private String name;
17
18     @Column(nullable = false, unique = true)
19     private String email;
20
21     @Column(nullable = false)
22     private String password;
23
24     @Enumerated(EnumType.STRING)
25     private Role role; // STUDENT, EMPLOYER, ADMIN
26
27     private String phone;
28     private String bio;
29     private String skills;
30     private String profilePhoto;
31 }

```

Listing 7.2: User.java – JPA Entity

8.3 Job Entity (Model)

```

1 package com.jobportal.model;
2
3 import jakarta.persistence.*;
4 import lombok.Data;
5 import java.time.LocalDate;
6
7 @Entity

```

```

8  @Table(name = "jobs")
9  @Data
10 public class Job {
11
12     @Id
13     @GeneratedValue(strategy = GenerationType.IDENTITY)
14     private Long id;
15
16     private String title;
17     private String description;
18     private String skillsRequired;
19     private String salaryRange;
20     private String location;
21     private String category;
22     private String jobType;
23     private LocalDate postedDate;
24
25     @ManyToOne
26     @JoinColumn(name = "employer_id")
27     private User employer;
28 }

```

Listing 7.3: Job.java – JPA Entity

8.4 Application Entity (Model)

```

1  package com.jobportal.model;
2
3  import jakarta.persistence.*;
4  import lombok.Data;
5  import java.time.LocalDate;
6
7  @Entity
8  @Table(name = "applications")
9  @Data
10 public class Application {
11
12     @Id

```

```

13     @GeneratedValue(strategy = GenerationType.IDENTITY)
14     private Long id;
15
16     @ManyToOne
17     @JoinColumn(name = "job_id")
18     private Job job;
19
20     @ManyToOne
21     @JoinColumn(name = "user_id")
22     private User candidate;
23
24     private String resumePath;
25     private String coverNote;
26
27     @Enumerated(EnumType.STRING)
28     private ApplicationStatus status; // APPLIED, SHORTLISTED, REJECTED
29
30     private LocalDate appliedDate;
31 }

```

Listing 7.4: Application.java – JPA Entity

8.5 Spring Security Configuration

```

1 package com.jobportal.security;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
6 import org.springframework.security.crypto.password.PasswordEncoder;
7 import org.springframework.security.web.SecurityFilterChain;
8 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
9
10 @Configuration
11 public class SecurityConfig {
12
13     @Bean
14     public SecurityFilterChain filterChain(HttpSecurity http)

```

```

15         throws Exception {
16     http
17         .authorizeHttpRequests(auth -> auth
18             .requestMatchers("/register", "/login",
19                 "/css/**", "/js/**", "/images/**").permitAll()
20             .requestMatchers("/employer/**").hasRole("EMPLOYER")
21             .requestMatchers("/candidate/**").hasRole("STUDENT")
22             .requestMatchers("/admin/**").hasRole("ADMIN")
23             .anyRequest().authenticated()
24         )
25         .formLogin(form -> form
26             .loginPage("/login")
27             .defaultSuccessUrl("/dashboard", true)
28             .permitAll()
29         )
30         .logout(logout -> logout
31             .logoutSuccessUrl("/login")
32             .permitAll()
33         );
34     return http.build();
35 }
36
37 @Bean
38 public PasswordEncoder passwordEncoder() {
39     return new BCryptPasswordEncoder();
40 }
41 }

```

Listing 7.5: SecurityConfig.java – Role-Based Access Control

8.6 Authentication Controller

```

1 package com.jobportal.controller;
2
3 import com.jobportal.model.User;
4 import com.jobportal.service.UserService;
5 import jakarta.validation.Valid;
6 import org.springframework.beans.factory.annotation.Autowired;

```

```

7 import org.springframework.stereotype.Controller;
8 import org.springframework.ui.Model;
9 import org.springframework.validation.BindingResult;
10 import org.springframework.web.bind.annotation.*;
11
12 @Controller
13 public class AuthController {
14
15     @Autowired
16     private UserService userService;
17
18     @GetMapping("/register")
19     public String showRegisterForm(Model model) {
20         model.addAttribute("user", new User());
21         return "register";
22     }
23
24     @PostMapping("/register")
25     public String register(
26         @Valid @ModelAttribute("user") User user,
27         BindingResult result, Model model) {
28         if (result.hasErrors()) {
29             return "register";
30         }
31         userService.registerUser(user);
32         return "redirect:/login";
33     }
34
35     @GetMapping("/login")
36     public String showLoginForm() {
37         return "login";
38     }
39 }

```

Listing 7.6: AuthController.java – Register and Login

8.7 Job Controller

```

1 package com.jobportal.controller;
2
3 import com.jobportal.model.Job;
4 import com.jobportal.service.JobService;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.stereotype.Controller;
7 import org.springframework.ui.Model;
8 import org.springframework.web.bind.annotation.*;
9
10 @Controller
11 @RequestMapping("/jobs")
12 public class JobController {
13
14     @Autowired
15     private JobService jobService;
16
17     @GetMapping
18     public String listJobs(
19         @RequestParam(required = false) String query,
20         @RequestParam(required = false) String location,
21         Model model) {
22         model.addAttribute("jobs",
23             jobService.searchJobs(query, location));
24         return "job-list";
25     }
26
27     @GetMapping("/{id}")
28     public String jobDetail(@PathVariable Long id, Model model) {
29         model.addAttribute("job", jobService.getJobById(id));
30         return "job-detail";
31     }
32
33     @PostMapping("/post")
34     public String postJob(@ModelAttribute Job job) {
35         jobService.saveJob(job);
36         return "redirect:/employer/dashboard";
37     }
38
39     @PostMapping("/delete/{id}")

```

```

40     public String deleteJob(@PathVariable Long id) {
41         jobService.deleteJob(id);
42         return "redirect:/employer/dashboard";
43     }
44 }

```

Listing 7.7: JobController.java – Job CRUD Operations

8.8 Application Controller

```

1  package com.jobportal.controller;
2
3  import com.jobportal.model.ApplicationStatus;
4  import com.jobportal.service.ApplicationService;
5  import org.springframework.beans.factory.annotation.Autowired;
6  import org.springframework.stereotype.Controller;
7  import org.springframework.ui.Model;
8  import org.springframework.web.bind.annotation.*;
9  import org.springframework.web.multipart.MultipartFile;
10
11  @Controller
12  @RequestMapping("/applications")
13  public class ApplicationController {
14
15      @Autowired
16      private ApplicationService applicationService;
17
18      @PostMapping("/apply/{jobId}")
19      public String applyForJob(@PathVariable Long jobId,
20                              @RequestParam MultipartFile resume,
21                              @RequestParam String coverNote) {
22          applicationService.applyForJob(jobId, resume, coverNote);
23          return "redirect:/candidate/dashboard";
24      }
25
26      @PostMapping("/update-status/{id}")
27      public String updateStatus(@PathVariable Long id,
28                               @RequestParam ApplicationStatus status) {

```

```

29         applicationService.updateStatus(id, status);
30         return "redirect:/employer/dashboard";
31     }
32
33     @GetMapping("/my")
34     public String myApplications(Model model) {
35         model.addAttribute("applications",
36             applicationService.getMyApplications());
37         return "candidate-dashboard";
38     }
39 }

```

Listing 7.8: ApplicationController.java – Apply and Track

8.9 User Service

```

1 package com.jobportal.service;
2
3 import com.jobportal.model.User;
4 import com.jobportal.repository.UserRepository;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.security.crypto.password.PasswordEncoder;
7 import org.springframework.stereotype.Service;
8
9 @Service
10 public class UserService {
11
12     @Autowired
13     private UserRepository userRepository;
14
15     @Autowired
16     private PasswordEncoder passwordEncoder;
17
18     public void registerUser(User user) {
19         user.setPassword(
20             passwordEncoder.encode(user.getPassword()));
21         userRepository.save(user);
22     }

```



```

23
24     public User findByEmail(String email) {
25         return userRepository.findByEmail(email)
26             .orElseThrow(() ->
27                 new RuntimeException("User not found: " + email));
28     }
29 }

```

Listing 7.9: UserService.java – Business Logic

8.10 Job Repository

```

1  package com.jobportal.repository;
2
3  import com.jobportal.model.Job;
4  import com.jobportal.model.User;
5  import org.springframework.data.jpa.repository.JpaRepository;
6  import org.springframework.data.jpa.repository.Query;
7  import org.springframework.data.repository.query.Param;
8  import java.util.List;
9
10 public interface JobRepository extends JpaRepository<Job, Long> {
11
12     List<Job> findByEmployer(User employer);
13
14     @Query("SELECT j FROM Job j WHERE " +
15         "(:query IS NULL OR LOWER(j.title) LIKE " +
16         "LOWER(CONCAT('%', :query, '%'))) AND " +
17         "(:location IS NULL OR LOWER(j.location) LIKE " +
18         "LOWER(CONCAT('%', :location, '%'))))")
19     List<Job> searchJobs(@Param("query") String query,
20         @Param("location") String location);
21
22     List<Job> findByCategory(String category);
23
24     List<Job> findByJobType(String jobType);
25 }

```

8.11 Application Properties Configuration

```
1 # Database Configuration
2 spring.datasource.url=jdbc:MongoDB://localhost:8081/jobportal
3 spring.datasource.username=root
4 spring.datasource.password=yourpassword
5 spring.datasource.driver-class-name=com.MongoDB.cj.jdbc.Driver
6
7 # JPA / Hibernate
8 spring.jpa.hibernate.ddl-auto=update
9 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MongoDB8Dialect
10 spring.jpa.show-sql=true
11
12 # Server
13 server.port=8080
14
15 # File Upload
16 spring.servlet.multipart.max-file-size=10MB
17 spring.servlet.multipart.max-request-size=10MB
18
19 # Thymeleaf
20 spring.thymeleaf.cache=false
```

Listing 7.11: application.properties – Database and Server Config

8.12 JSearch API Integration (JavaScript)

```
1 async function fetchJSearchJobs(query, location) {
2     const url =
3         'https://jsearch.p.rapidapi.com/search?' +
4         'query=${encodeURIComponent(query)}&' +
5         'location=${encodeURIComponent(location)}&page=1';
6 }
```

```

7   const options = {
8       method: 'GET',
9       headers: {
10          'x-rapidapi-key': 'YOUR_RAPIDAPI_KEY',
11          'x-rapidapi-host': 'jsearch.p.rapidapi.com'
12      }
13  };
14
15  try {
16      document.getElementById('spinner').style.display = 'block';
17      const response = await fetch(url, options);
18      const data = await response.json();
19      displayJobs(data.data);
20  } catch (error) {
21      document.getElementById('error-msg').innerText =
22          'Failed to load external jobs. Try again.';
23  } finally {
24      document.getElementById('spinner').style.display = 'none';
25  }
26 }
27
28 function displayJobs(jobs) {
29     const container = document.getElementById('job-cards');
30     jobs.forEach(job => {
31         const card = `
32         <div class="job-card">
33             <h4>${job.job_title}</h4>
34             <p>${job.employer_name} - ${job.job_city}</p>
35             <p>${job.job_employment_type}</p>
36             <a href="${job.job_apply_link}" target="_blank"
37                 class="btn">Apply Now</a>
38         </div>`;
39         container.innerHTML += card;
40     });
41 }

```

Listing 7.12: jsearch.js – Live Job Listings from RapidAPI

8.13 EmailJS Integration (JavaScript)

```
1 emailjs.init('YOUR_EMAILJS_PUBLIC_KEY');
2
3 function sendConfirmationEmail(candidateName, jobTitle,
4                               companyName, toEmail) {
5     const templateParams = {
6         candidate_name: candidateName,
7         job_title: jobTitle,
8         company_name: companyName,
9         to_email: toEmail
10    };
11
12    emailjs.send(
13        'YOUR_SERVICE_ID',
14        'YOUR_TEMPLATE_ID',
15        templateParams
16    )
17    .then(function(response) {
18        showToast('Application submitted successfully!', 'success');
19    }, function(error) {
20        showToast('Submission failed. Please try again.', 'error');
21    });
22 }
23
24 document.getElementById('applyForm')
25     .addEventListener('submit', function(e) {
26         e.preventDefault();
27         const name = document.getElementById('name').value;
28         const job = document.getElementById('jobTitle').value;
29         const company = document.getElementById('company').value;
30         const email = document.getElementById('email').value;
31         sendConfirmationEmail(name, job, company, email);
32         this.submit();
33     });
```

Listing 7.13: emailjs-integration.js – Auto Email on Application

8.14 Global Exception Handler

```
1 package com.jobportal.exception;
2
3 import org.springframework.ui.Model;
4 import org.springframework.web.bind.annotation.ControllerAdvice;
5 import org.springframework.web.bind.annotation.ExceptionHandler;
6
7 @ControllerAdvice
8 public class GlobalExceptionHandler {
9
10     @ExceptionHandler(RuntimeException.class)
11     public String handleRuntimeException(
12         RuntimeException ex, Model model) {
13         model.addAttribute("errorMessage", ex.getMessage());
14         return "error";
15     }
16
17     @ExceptionHandler(Exception.class)
18     public String handleGeneralException(
19         Exception ex, Model model) {
20         model.addAttribute("errorMessage",
21             "An unexpected error occurred.");
22         return "error";
23     }
24 }
```

Listing 7.14: GlobalExceptionHandler.java – @ControllerAdvice